

FreeRTOS软件定时器 基于STM32

文章目录

[一、软件定时器的基本概念](#)

[二、软件定时器应用场景](#)

[三、软件定时器的精度](#)

[四、软件定时器的运作机制](#)

[五、软件定时器函数接口讲解](#)

[1.软件定时器创建函数 xTimerCreate\(\)](#)

[2.软件定时器启动函数 xTimerStart\(\)](#)

[3.软件定时器停止函数 xTimerStop\(\)](#)

[4.软件定时器任务](#)

[5.软件定时器删除函数 xTimerDelete\(\)](#)

[五、软件定时器实验](#)

[六、实验现象](#)

一、软件定时器的基本概念

定时器，是指从指定的时刻开始，经过一个指定时间，然后触发一个超时事件，用户可以自定义定时器的周期与频率。类似生活中的[闹钟](#)，我们可以设置闹钟每天什么时候响，还能设置响的次数，是响一次还是每天都响。

定时器有硬件定时器和软件定时器之分：

硬件定时器是芯片本身提供的定时功能。一般是由外部[晶振](#)提供给芯片输入时钟，芯片向软件模块提供一组配置寄存器，接受控制输入，到达设定时间值后芯片中断控制器产生时钟中断。硬件定时器的精度一般很高，可以达到纳秒级别，并且是中断触发方式。

软件定时器，软件定时器是由操作系统提供的一类系统接口，它构建在硬件定时器基础之上，使系统能够提供不受硬件定时器资源限制的定时器服务，它实现的功能与硬件定时器也是类似的。

使用硬件定时器时，每次在定时时间到达之后就会自动触发一个中断，用户在中断中处理信息；而使用软件定时器时，需要我们在创建软件定时器时指定时间到达后要调用的函数（也称超时函数/回调函数，为了统一，下文均用回调函数描述），在回调函数中处理信息。

注意：软件定时器回调函数的上下文是任务，下文所说的定时器均为软件定时器。

软件定时器在被创建之后，当经过设定的时钟计数值后会触发用户定义的回调函数。定时精度与系统时钟的周期有关。一般系统利用[SysTick](#)作为软件定时器的基础时钟，软件定时器的回调函数类似硬件的中断服务函数，所以，回调函数也要快进快出，而且回调函数中不能有任何阻塞任务运行的情况（软件定时器回调函数的上下文环境是任务），比如 `vTaskDelay()` 以及其它能阻塞任务运行的函数，两次触发回调函数的时间间隔 `xTimerPeriodInTicks` 叫定时器的定时周期。

FreeRTOS 操作系统提供软件定时器功能，软件定时器的使用相当于扩展了定时器的数量，允许创建更多的定时业务。FreeRTOS 软件定时器功能上支持：

裁剪：能通过宏关闭软件定时器功能。

软件定时器创建。

软件定时器启动。

软件定时器停止。

软件定时器复位。

软件定时器删除。

FreeRTOS 提供的软件定时器支持单次模式和周期模式，单次模式和周期模式的定时时间到之后都会调用软件定时器的回调函数，用户可以在回调函数中加入要执行的工程代码。

单次模式：当用户创建了定时器并启动了定时器后，定时时间到了，只执行一次回调函数之后就将该定时器进入休眠状态，不再重新执行。

周期模式：这个定时器会按照设置的定时时间循环执行回调函数，直到用户将定时器删除。具体看图1

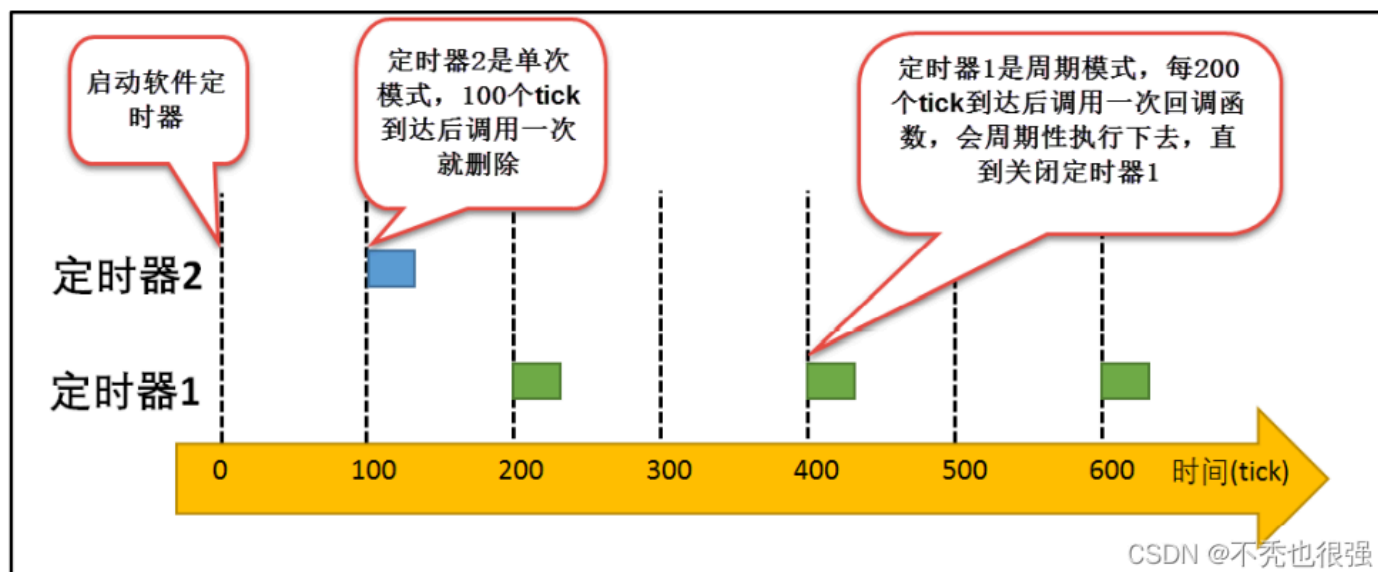


图1软件定时器的单次模式与周期模式

FreeRTOS 通过一个 `prvTimerTask` 任务（也叫守护任务 Daemon）管理软定时器，它是在启动调度器时自动创建的，为了满足用户定时需求。`prvTimerTask` 任务会在其执行期间检查用户启动的时间周期溢出的定时器，并调用其回调函数。只有设置 `FreeRTOSConfig.h` 中的宏定义 `configUSE_TIMERS` 设置为 1，将相关代码编译进来，才能正常使用软件定时器相关功能。

二、软件定时器应用场景

在很多应用中，我们需要一些定时器任务，硬件定时器受硬件的限制，数量上不足以满足用户的实际需求，无法提供更多的定时器，那么可以采用软件定时器来完成，由软件定时器代替硬件定时器任务。但需要注意的是软件定时器的精度是无法和硬件定时器相比的，而且在软件定时器的定时过程中是极有可能被其它中断所打断，因为软件定时器的执行上下文环境是任务。所以，软件定时器更适用于对时间精度要求不高的任务，一些辅助型的任务。

三、软件定时器的精度

在操作系统中，通常软件定时器以系统节拍周期为计时单位。系统节拍是系统的心跳节拍，表示系统时钟的频率，就类似人的心跳，1s 能跳动多少下，系统节拍配置为 `configTICK_RATE_HZ`，该宏在 `FreeRTOSConfig.h` 中有定义，默认是 1000。那么系统的时钟节拍周期就为 1ms（1s 跳动 1000 下，每一下就为 1ms）。软件定时器的所定时数值必须是这个节拍周期的整数倍，例如节拍周期是 10ms，那么上层软件定时器定时数值只能是 10ms，20ms，100ms 等，而不能取值为 15ms。由于节拍定义了系统中定时器能够分辨的精确度，系统可以根据实际系统 CPU 的处理能力和实时性需求设置合适的数值，系统节拍周期的值越小，精度越高，但是系统开销也将越大，因为这代表在 1 秒中系统进入时钟中断的次数也就越多。

四、软件定时器的运作机制

软件定时器是可选的系统资源，在创建定时器的时候会分配一块内存空间。当用户创建并启动一个软件定时器时，FreeRTOS 会根据当前系统时间及用户设置的定时确定该定时器唤醒时间，并将该定时器控制块挂入软件定时器列表，FreeRTOS 中采用两个定时器列表维护软件定时器，`pxCurrentTimerList` 与 `pxOverflowTimerList` 是列表指针，在初始化的时候分别指向 `xActiveTimerList1` 与 `xActiveTimerList2`

软件定时器用到的列表

```
1. PRIVILEGED_DATA static List_t xActiveTimerList1;
2. PRIVILEGED_DATA static List_t xActiveTimerList2;
3. PRIVILEGED_DATA static List_t *pxCurrentTimerList;
4. PRIVILEGED_DATA static List_t *pxOverflowTimerList;
```

`xCurrentTimerList`：系统新创建并激活的定时器都会以超时时间升序的方式插入到 `pxCurrentTimerList` 列表中。系统在定时器任务中扫描 `pxCurrentTimerList` 中的第一个定时器，看是否已超时，若已经超时了则调用软件定时器回调函数。否则将定时器任务挂起，因为定时时间是升序插入软件定时器列表的，列表中第一个定时器的定时时间都还没到的话，那后面的定时器定时时间自然没到。

pxOverflowTimerList 列表是在软件定时器溢出的时候使用，作用与 pxCurrentTimerList 一致。同时，FreeRTOS 的软件定时器还有采用消息队列进行通信，利用“定时器命令队列”向软件定时器任务发送一些命令，任务在接收到命令就会去处理命令对应的程序，比如启动定时器，停止定时器等。假如定时器任务处于阻塞状态，我们又需要马上再添加一个软件定时器的话，就是采用这种消息队列命令的方式进行添加，才能唤醒处于等待状态的定时器任务，并且在任务中将新添加的软件定时器添加到软件定时器列表中，所以，在定时器启动函数中，FreeRTOS 是采用队列的方式发送一个消息给软件定时器任务，任务被唤醒从而执行接收到的命令。

例如：系统当前时间 xTimeNow 值为 0，注意：xTimeNow 其实是一个局部变量，是根据 xTaskGetTickCount()函数获取的，实际它的值就是全局变量 xTickCount 的值，下文都采用它表示当前系统时间。在当前系统中已经创建并启动了 1 个定时器 Timer1；系统继续运行，当系统的时间 xTimeNow 为 20 的时候，用户创建并且启动一个定时时间为 100 的定时器 Timer2，此时 Timer2 的溢出时间 xTicksToWait 就为定时时间 + 系统当前时间 (100+20=120)，然后将 Timer2 按 xTicksToWait 升序插入软件定时器列表中；假设当前系统时间 xTimeNow 为 40 的时候，用户创建并且启动了一个定时时间为 50 的定时器 Timer3，那么此时 Timer3 的溢出时间 xTicksToWait 就为 40+50=90，同样安装 xTicksToWait 的数值升序插入软件定时器列表中，在定时器链表中插入过程具体见图 2。同理创建并且启动在已有的两个定时器中间的定时器也是一样的，具体见图3。

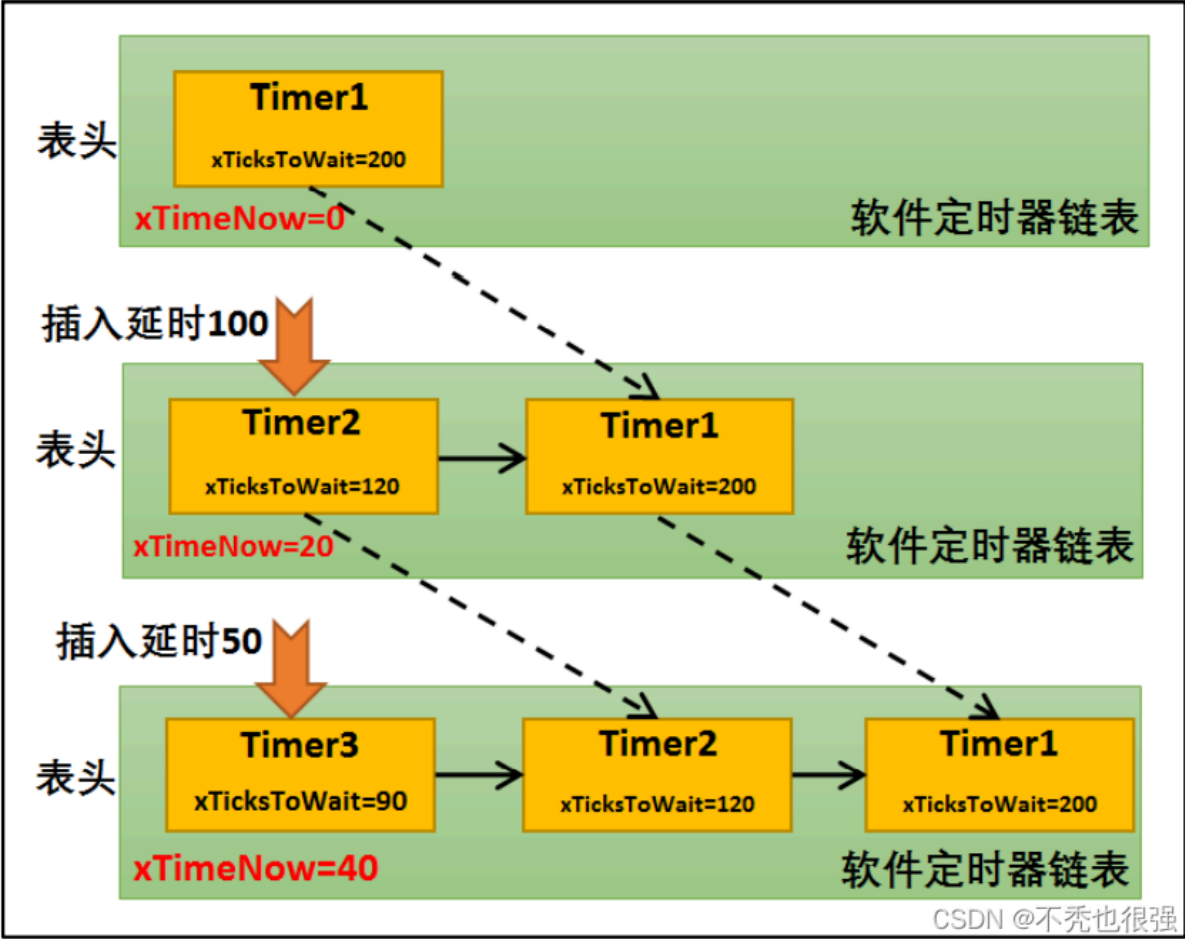


图2 定时器链表示意图1

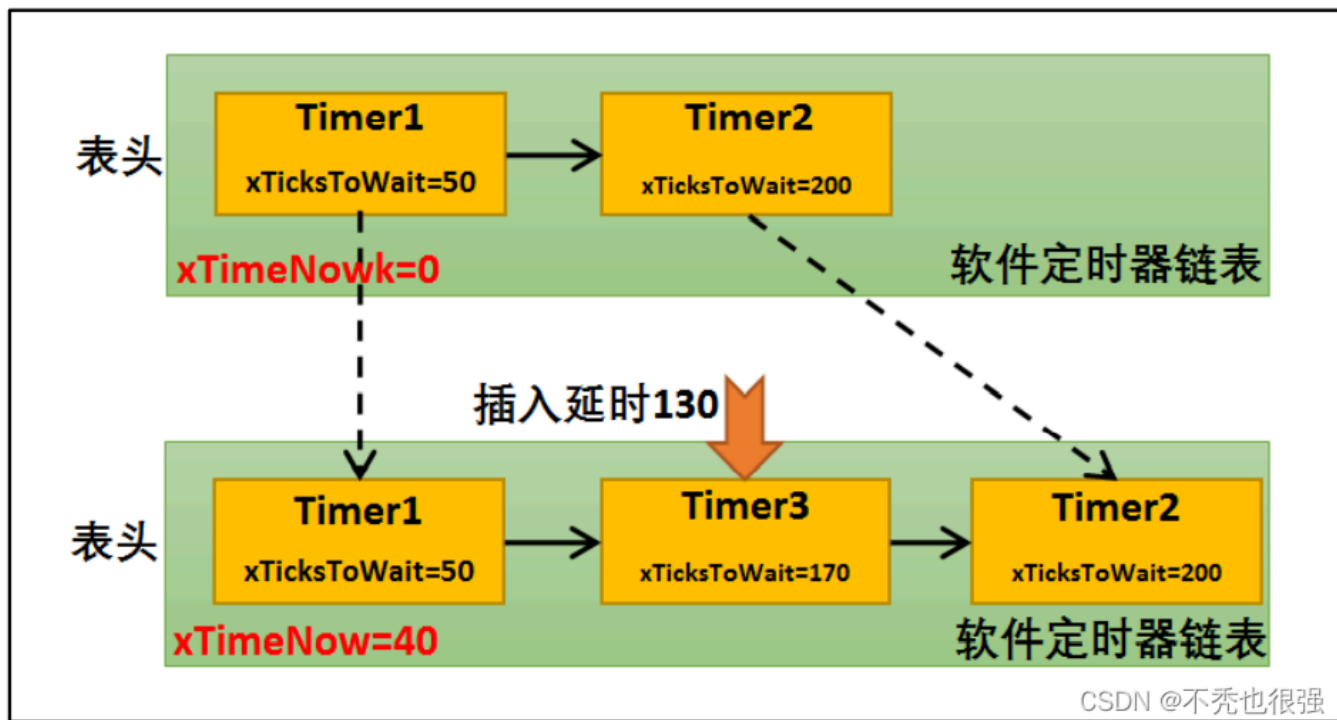


图 3 定时器链表示意图 2

那么系统如何处理软件定时器列表？系统在不断运行，而 `xTimeNow` (`xTickCount`) 随着 `SysTick` 的触发一直在增长（每一次硬件定时器中断来临，`xTimeNow` 变量会加 1），在软件定时器任务运行的时候会获取下一个要唤醒的定时器，比较当前系统时间 `xTimeNow` 是否大于或等于下一个定时器唤醒时间 `xTicksToWait`，若大于则表示已经超时，定时器任务将会调用对应定时器的回调函数，否则将软件定时器任务挂起，直至下一个要唤醒的软件定时器时间到来或者接收到命令消息。

使用软件定时器时候要注意以下几点：

软件定时器的回调函数中应快进快出，绝对不允许使用任何可能引软件定时器起任务挂起或者阻塞的 API 接口，在回调函数中也绝对不允许出现死循环。

软件定时器使用了系统的一个队列和一个任务资源，软件定时器任务的优先级默认为 `configTIMER_TASK_PRIORITY`，为了更好响应，该优先级应设置为所有任务中最高的优先级。创建单次软件定时器，该定时器超时执行完回调函数后，系统会自动删除该软件定时器，并回收资源。

定时器任务的堆栈大小默认为 `configTIMER_TASK_STACK_DEPTH` 个字节。

五、软件定时器函数接口讲解

软件定时器的功能是在定时器任务（或者叫定时器守护任务）中实现的。软件定时器的很多 API 函数通过一个名字叫“定时器命令队列”的队列来给定时器守护任务发送命令。该定时器命令队列由 RTOS 内核提供，且应用程序不能够直接访问，其消息队列的长度由宏 `configTIMER_QUEUE_LENGTH` 定义，下面就讲解一些常用的软件定时器函数接口。

1. 软件定时器创建函数 `xTimerCreate()`

软件定时器与 FreeRTOS 内核其他资源一样，需要创建才允许使用的，FreeRTOS 为我们提供了两种创建方式，一种是动态创建软件定时器 `xTimerCreate()`，另一种是静态创建方式 `xTimerCreateStatic()`，因为创建过程基本差不多，所以在这里我们只讲解动态创建方式。

`xTimerCreate()` 用于创建一个软件定时器，并返回一个句柄。要想使用该函数必须在头文件 `FreeRTOSConfig.h` 中把宏 `configUSE_TIMERS` 和 `configSUPPORT_DYNAMIC_ALLOCATION` 均定义为 1（`configSUPPORT_DYNAMIC_ALLOCATION` 在 `FreeRTOS.h` 中默认定义为 1），并且需要把 `FreeRTOS/source/timers.c` 这个 C 文件添加到工程中。每一个软件定时器只需要很少的 RAM 空间来保存其的状态。如果使用函数 `xTimeCreate()` 来创建一个软件定时器，那么需要的 RAM 是动态分配的。如果使用函数 `xTimeCreateStatic()` 来创建一个事件组，那么需要的 RAM 是静态分配的。软件定时器在创建成功后是处于休眠状态的，可以使用 `xTimerStart()`、`xTimerReset()`、`xTimerStartFromISR()`、`xTimerResetFromISR()`、`xTimerChangePeriod()` 和 `xTimerChangePeriodFromISR()` 这些函数将其状态转换为活跃态。

`xTimerCreate()` 使用实例

```
1.
2. static TimerHandle_t Swtmr1_Handle = NULL; /* 软件定时器句柄 */
3. static TimerHandle_t Swtmr2_Handle = NULL; /* 软件定时器句柄 */
4. static void AppTaskCreate(void)
5. {
```

```

6. taskENTER_CRITICAL();          //进入临界区
7.
8. /*****
9. * 创建软件周期定时器
10. * 函数原型
11. * TimerHandle_t xTimerCreate(          const char * const pcTimerName,
12.                                     const TickType_t xTimerPeriodInTicks,
13.                                     const UBaseType_t uxAutoReload,
14.                                     void * const pvTimerID,
15.                                     TimerCallbackFunction_t pxCallbackFunction )
16. * @uxAutoReload : pdTRUE为周期模式，pdFALSE为单次模式
17. * 单次定时器，周期(1000个时钟节拍)，周期模式
18. *****/
19. Swtmr1_Handle=xTimerCreate((const char*          )"AutoReloadTimer",
20.                             (TickType_t          )1000,/* 定时器周期 1000(tick) */
21.                             (UBaseType_t          )pdTRUE,/* 周期模式 */
22.                             (void*                )1,/* 为每个计时器分配一个索引的唯一 ID */
23.                             (TimerCallbackFunction_t)Swtmr1_Callback);
24. if(Swtmr1_Handle != NULL)
25. {
26.     /*****
27.     * xTicksToWait:如果在调用xTimerStart()时队列已满，则以tick为单位指定调用任务应保持
28.     * 在Blocked(阻塞)状态以等待start命令成功发送到timer命令队列的时间。
29.     * 如果在启动调度程序之前调用xTimerStart()，则忽略xTicksToWait。在这里设置等待时间为0.
30.     *****/
31.     xTimerStart(Swtmr1_Handle, 0);          //开启周期定时器
32. }
33. /*****
34. * 创建软件周期定时器
35. * 函数原型
36. * TimerHandle_t xTimerCreate(          const char * const pcTimerName,
37.                                     const TickType_t xTimerPeriodInTicks,
38.                                     const UBaseType_t uxAutoReload,
39.                                     void * const pvTimerID,
40.                                     TimerCallbackFunction_t pxCallbackFunction )
41. * @uxAutoReload : pdTRUE为周期模式，pdFALSE为单次模式
42. * 单次定时器，周期(5000个时钟节拍)，单次模式
43. *****/
44. Swtmr2_Handle=xTimerCreate((const char*          )"OneShotTimer",
45.                             (TickType_t          )5000,/* 定时器周期 5000(tick) */
46.                             (UBaseType_t          )pdFALSE,/* 单次模式 */
47.                             (void*                )2,/* 为每个计时器分配一个索引的唯一 ID */
48.                             (TimerCallbackFunction_t)Swtmr2_Callback);
49. if(Swtmr2_Handle != NULL)
50. {
51.     /*****
52.     * xTicksToWait:如果在调用xTimerStart()时队列已满，则以tick为单位指定调用任务应保持
53.     * 在Blocked(阻塞)状态以等待start命令成功发送到timer命令队列的时间。
54.     * 如果在启动调度程序之前调用xTimerStart()，则忽略xTicksToWait。在这里设置等待时间为0.
55.     *****/
56.     xTimerStart(Swtmr2_Handle, 0);          //开启周期定时器
57. }
58. vTaskDelete(AppTaskCreate_Handle); //删除AppTaskCreate任务
59.
60. taskEXIT_CRITICAL();          //退出临界区
61. }
62. static void Swtmr1_Callback(void* parameter)
63. {
64.     /* 软件定时器的回调函数，用户自己实现 */
65. }
66.
67. static void Swtmr2_Callback(void* parameter)
68. {
69.     /* 软件定时器的回调函数，用户自己实现 */
70. }
71. }

```



2.软件定时器启动函数 xTimerStart()

xTimerStart()

如果是认真看上面 xTimerCreate()函数使用实例的同学应该就发现了，这个软件定时器 启动函数 xTimerStart()在上面的实例中有用到过，前一小节已经说明了，软件定时器在创建完成的时候是处于休眠状态的，需要用 FreeRTOS 的相关函数将软件定时器活动起来，而 xTimerStart()函数就是可以让处于休眠的定时器开始工作。我们知道，在系统开始运行的时候，系统会帮我们自动创建一个软件定时器任务（prvTimerTask），在这个任务中，如果暂时没有运行中的定时器，任务会进入阻塞态等待命令，而我们的启动函数就是通过“定时器命令队列”向定时器任务发送一个启动命令，定时器任务获得命令就解除阻塞，然后执行启动软件定时器命令。

xTimerStartFromISR()

当然除在任务启动软件定时器之外，还有在中断中启动软件定时器的函数 xTimerStartFromISR()。xTimerStartFromISR()是函数 xTimerStart()的中断版本，用于启动一个先前由函数 xTimerCreate() / xTimerCreateStatic()创建的软件定时器。

xTimerStartFromISR()函数说明


```
12.         TimerCallbackFunction_t pxCallbackFunction )
13.     * @uxAutoReload : pdTRUE为周期模式，pdFALSE为单次模式
14.     * 单次定时器，周期(1000个时钟节拍)，周期模式
15.     *****/
16.     Swtmr1_Handle=xTimerCreate((const char*          )"AutoReloadTimer",
17.                               (TickType_t          )1000,/* 定时器周期 1000(tick) */
18.                               (UBaseType_t         )pdTRUE,/* 周期模式 */
19.                               (void*                )1,/* 为每个计时器分配一个索引的唯一ID */
20.                               (TimerCallbackFunction_t)Swtmr1_Callback);
21.     if(Swtmr1_Handle != NULL)
22.     {
23.         /*****
24.         * xTicksToWait:如果在调用xTimerStart()时队列已满，则以tick为单位指定调用任务应保持
25.         * 在Blocked(阻塞)状态以等待start命令成功发送到timer命令队列的时间。
26.         * 如果在启动调度程序之前调用xTimerStart()，则忽略xTicksToWait。在这里设置等待时间为0.
27.         *****/
28.         xTimerStart(Swtmr1_Handle,0);          //开启周期定时器
29.     }
30.     static void test_task(void* parameter)
31.     {
32.         while (1) {
33.             /* 用户自己实现任务代码 */
34.             xTimerStop(Swtmr1_Handle,0); //停止定时器
35.         }
36.     }
37. }
```



xTimerStopFromISR()

xTimerStopFromISR()是函数 xTimerStop()的中断版本，用于停止一个正在运行的软件 定时器，让其进入休眠态，实现过程也是通过“定时器命令队列”向软件定时器任务发送 停止命令。xTimerStopFromISR()函数说明

函数原	BaseType_t xTimerStopFromISR(TimerHandle_t xTimer, BaseType_t *pxHigherPriorityTaskWoken);		CSDN @不秃也很强
型			
功能	在中断中停止一个软件定时器，让其进入休眠态。		
形参	xTimer	软件定时器句柄。	
	pxHigherPriorityTaskWoken	定时器守护任务的大部分时间都在阻塞态等待定时器命令队列的命令。调用函数 xTimerStopFromISR()将会往定时器的命令队列发送一个停止命令，这很有可能会将定时器任务从阻塞态移除。如果调用函数 xTimerStopFromISR()让定时器任务脱离阻塞态，且定时器守护任务的优先级大于或者等于当前被中断的任务的优先级，那么 pxHigherPriorityTaskWoken 的值会在函数 xTimerStopFromISR()内部设置为 pdTRUE，然后在中断退出之前执行一次上下文切换。	
返回值	如果停止命令在超时时间之前无法成功地发送到定时器命令队列则返回 pdFAILE，成功发送则返回 pdPASS。软件定时器成功发送的命令是否真正的被执行也还要看定时器守护任务的优先级，其优先级由宏 configTIMER_TASK_PRIORITY 定义。		
	CSDN @不秃也很强		

xTimerStopFromISR()函数应用举例

```
1. void vAnExampleInterruptServiceRoutine( void )
2. {
3.     BaseType_t xHigherPriorityTaskWoken = pdFALSE;
4.
5.     if ( xTimerStopFromISR(xTimer,&xHigherPriorityTaskWoken)!=pdPASS ) {
6.         /* 软件定时器停止命令没有成功执行 */
7.     }
8.
9.
10.    if ( xHigherPriorityTaskWoken != pdFALSE ) {
11.        /* 执行上下文切换 */
12.    }
13. }
```



4.软件定时器任务

我们知道，软件定时器回调函数运行的上下文环境是任务，那么软件定时器任务是在 干什么的呢？如何创建的呢？下面跟我一步步来分析软件定时器的工作过程。软件定时器任务是在系统开始调度（vTaskStartScheduler()函数）的时候就被创建的，前提是宏定义 configUSE_TIMERS 开启，具体见代码清单 21-12 加粗部分，在 xTimerCreateTimerTask()函数里面就是创建了一个软件定时器任务，就跟我们创建任务一样，支持动态与静态创建，我们暂时看动态创建的即可，

5.软件定时器删除函数 xTimerDelete()

xTimerDelete()用于删除一个已经被创建成功的软件定时器，删除之后就无法使用该定时器，并且定时器相应的资源也会被系统回收释放。要想使函数 xTimerStop()必须在头文件 FreeRTOSConfig.h 中把宏 configUSE_TIMERS 定义为 1。

xTimerDelete()函数说明

函数原型	#define xTimerDelete(xTimer, xTicksToWait) xTimerGenericCommand((xTimer), tmrCOMMAND_DELETE, 0U, NULL, (xTicksToWait))	
功能	删除一个已经被创建成功的软件定时器。	
形参	xTimer	软件定时器句柄。
	xBlockTime	用户指定的超时时间，单位为系统节拍周期(即 tick)，如果在 FreeRTOS 调度器开启之前调用 xTimerStart()，该形参将不起作用。
返回值	如果删除命令在超时时间之前无法成功地发送到定时器命令队列则返回 pdFAILE，成功发送则返回 pdPASS。	

CSDN @不秃也很强

从软件定时器删除函数 xTimerDelete()的原型可以看出，删除一个软件定时器也是在软件定时器任务中删除，调用 xTimerDelete()将删除软件定时器的命令发送给软件定时器任务，软件定时器任务在接收到删除的命令之后就进行删除操作，该函数的使用方法很简单。

xTimerDelete()使用实例

```
1. static void test_task(void* parameter)
2. {
3.     while (1) {
4.         /* 用户自己实现任务代码 */
5.         xTimerDelete(Swtmr1_Handle, 0); //删除软件定时器
6.     }
7. }
```

五、软件定时器实验

软件定时器实验是在 FreeRTOS 中创建了两个软件定时器，其中一个软件定时器是单次模式，5000 个 tick 调用一次回调函数，另一个软件定时器是周期模式，1000 个 tick 调用一次回调函数，在回调函数中输出相关信息。

```
1.
2. /* FreeRTOS头文件 */
3. #include "FreeRTOS.h"
4. #include "task.h"
5. #include "event_groups.h"
6. /* 开发板硬件bsp头文件 */
7. #include "bsp_led.h"
8. #include "bsp_usart.h"
9. #include "bsp_key.h"
10. /****** 任务句柄 *****/
11. /*
12. * 任务句柄是一个指针，用于指向一个任务，当任务创建好之后，它就具有了一个任务句柄
13. * 以后我们要想操作这个任务都需要通过这个任务句柄，如果是自身的任务操作自己，那么
14. * 这个句柄可以为NULL。
15. */
16. static TaskHandle_t AppTaskCreate_Handle = NULL; /* 创建任务句柄 */
17.
18. /****** 内核对象句柄 *****/
19. /*
20. * 信号量，消息队列，事件标志组，软件定时器这些都属于内核的对象，要想使用这些内核
21. * 对象，必须先创建，创建成功之后会返回一个相应的句柄。实际上就是一个指针，后续我
```



```

22. * 们就可以通过这个句柄操作这些内核对象。
23. *
24. * 内核对象说白了就是一种全局的数据结构，通过这些数据结构我们可以实现任务间的通信，
25. * 任务间的事件同步等各种功能。至于这些功能的实现我们是通过调用这些内核对象的函数
26. * 来完成的
27. *
28. */
29. static TimerHandle_t Swtmr1_Handle = NULL; /* 软件定时器句柄 */
30. static TimerHandle_t Swtmr2_Handle = NULL; /* 软件定时器句柄 */
31. /***** 全局变量声明 *****/
32. /*
33. * 当我们在写应用程序的时候，可能需要用到一些全局变量。
34. */
35. static uint32_t TmrCb_Count1 = 0; /* 记录软件定时器1回调函数执行次数 */
36. static uint32_t TmrCb_Count2 = 0; /* 记录软件定时器2回调函数执行次数 */
37.
38. /***** 宏定义 *****/
39. /*
40. * 当我们在写应用程序的时候，可能需要用到一些宏定义。
41. */
42.
43. /*
44. *****/
45. * 函数声明
46. *****/
47. /*
48. static void AppTaskCreate(void);/* 用于创建任务 */
49.
50. static void Swtmr1_Callback(void* parameter);
51. static void Swtmr2_Callback(void* parameter);
52.
53. static void BSP_Init(void);/* 用于初始化板载相关资源 */
54.
55. /*****
56. * @brief 主函数
57. * @param 无
58. * @retval 无
59. * @note 第一步：开发板硬件初始化
60. 第二步：创建APP应用任务
61. 第三步：启动FreeRTOS，开始多任务调度
62. *****/
63. int main(void)
64. {
65. BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
66.
67. /* 开发板硬件初始化 */
68. BSP_Init();
69.
70. printf("这是一个FreeRTOS软件定时器实验！\n");
71.
72. /* 创建AppTaskCreate任务 */
73. xReturn = xTaskCreate((TaskFunction_t )AppTaskCreate, /* 任务入口函数 */
74. (const char* )"AppTaskCreate", /* 任务名字 */
75. (uint16_t )512, /* 任务栈大小 */
76. (void* )NULL, /* 任务入口函数参数 */
77. (UBaseType_t )1, /* 任务的优先级 */
78. (TaskHandle_t* )&AppTaskCreate_Handle); /* 任务控制块指针 */
79.
80. /* 启动任务调度 */
81. if(pdPASS == xReturn)
82. vTaskStartScheduler(); /* 启动任务，开启调度 */
83. else
84. return -1;
85.
86. while(1); /* 正常不会执行到这里 */
87. }
88.
89. /*****
90. * @ 函数名 : AppTaskCreate
91. * @ 功能说明: 为了方便管理，所有的任务创建函数都放在这个函数里面
92. * @ 参数 : 无
93. * @ 返回值 : 无
94. *****/
95. static void AppTaskCreate(void)
96. {
97. taskENTER_CRITICAL(); //进入临界区
98.
99. /*****
100. * 创建软件周期定时器
101. * 函数原型
102. * TimerHandle_t xTimerCreate( const char * const pcTimerName,
103. const TickType_t xTimerPeriodInTicks,
104. const UBaseType_t uxAutoReload,
105. void * const pvTimerID,
106. TimerCallbackFunction_t pxCallbackFunction )
107. * @uxAutoReload : pdTRUE为周期模式，pdFALSE为单次模式
108. * 单次定时器，周期(1000个时钟节拍)，周期模式
109. *****/
110. Swtmr1_Handle=xTimerCreate((const char* )"AutoReloadTimer",
111. (TickType_t )1000, /* 定时器周期 1000(tick) */
112. (UBaseType_t )pdTRUE, /* 周期模式 */
113. (void* )1, /* 为每个计时器分配一个索引的唯一ID */
114. (TimerCallbackFunction_t)Swtmr1_Callback);
115. if(Swtmr1_Handle != NULL)
116. {
117. /*****
118. * xTicksToWait:如果在调用xTimerStart()时队列已满，则以tick为单位指定调用任务应保持
119. * 在Blocked(阻塞)状态以等待start命令成功发送到timer命令队列的时间。

```

```

120.     * 如果在启动调度程序之前调用xTimerStart(), 则忽略xTicksToWait。在这里设置等待时间为0。
121.     *****/
122.     xTimerStart(Swtmr1_Handle, 0);           //开启周期定时器
123. }
124. /*****/
125. * 创建软件周期定时器
126. * 函数原型
127. * TimerHandle_t xTimerCreate(           const char * const pcTimerName,
128.                                     const TickType_t xTimerPeriodInTicks,
129.                                     const UBaseType_t uxAutoReload,
130.                                     void * const pvTimerID,
131.                                     TimerCallbackFunction_t pxCallbackFunction )
132. * @uxAutoReload : pdTRUE为周期模式, pdFALSE为单次模式
133. * 单次定时器, 周期(5000个时钟节拍), 单次模式
134. *****/
135.     Swtmr2_Handle=xTimerCreate((const char*           )"OneShotTimer",
136.                               (TickType_t           )5000, /* 定时器周期 5000(tick) */
137.                               (UBaseType_t          )pdFALSE, /* 单次模式 */
138.                               (void*                )2, /* 为每个计时器分配一个索引的唯一ID */
139.                               (TimerCallbackFunction_t)Swtmr2_Callback);
140. if(Swtmr2_Handle != NULL)
141. {
142.     /*****/
143.     * xTicksToWait:如果在调用xTimerStart()时队列已满, 则以tick为单位指定调用任务应保持
144.     * 在Blocked(阻塞)状态以等待start命令成功发送到timer命令队列的时间。
145.     * 如果在启动调度程序之前调用xTimerStart(), 则忽略xTicksToWait。在这里设置等待时间为0。
146.     *****/
147.     xTimerStart(Swtmr2_Handle, 0);           //开启周期定时器
148. }
149.
150. vTaskDelete(AppTaskCreate_Handle); //删除AppTaskCreate任务
151.
152. taskEXIT_CRITICAL();           //退出临界区
153. }
154.
155. /*****/
156. * @ 函数名 : Swtmr1_Callback
157. * @ 功能说明: 软件定时器1 回调函数, 打印回调函数信息&当前系统时间
158. *           软件定时器请不要调用阻塞函数, 也不要进行死循环, 应尽快快出
159. * @ 参数 : 无
160. * @ 返回值 : 无
161. *****/
162. static void Swtmr1_Callback(void* parameter)
163. {
164.     TickType_t tick_num1;
165.
166.     TmrCb_Count1++;           /* 每回调一次加一 */
167.
168.     tick_num1 = xTaskGetTickCount(); /* 获取滴答定时器的计数值 */
169.
170.     LED1_TOGGLE;
171.
172.     printf("Swtmr1_Callback函数执行 %d 次\n", TmrCb_Count1);
173.     printf("滴答定时器数值=%d\n", tick_num1);
174. }
175.
176. /*****/
177. * @ 函数名 : Swtmr2_Callback
178. * @ 功能说明: 软件定时器2 回调函数, 打印回调函数信息&当前系统时间
179. *           软件定时器请不要调用阻塞函数, 也不要进行死循环, 应尽快快出
180. * @ 参数 : 无
181. * @ 返回值 : 无
182. *****/
183. static void Swtmr2_Callback(void* parameter)
184. {
185.     TickType_t tick_num2;
186.
187.     TmrCb_Count2++;           /* 每回调一次加一 */
188.
189.     tick_num2 = xTaskGetTickCount(); /* 获取滴答定时器的计数值 */
190.
191.     printf("Swtmr2_Callback函数执行 %d 次\n", TmrCb_Count2);
192.     printf("滴答定时器数值=%d\n", tick_num2);
193. }
194.
195.
196. /*****/
197. * @ 函数名 : BSP_Init
198. * @ 功能说明: 板级外设初始化, 所有板子上的初始化均可放在这个函数里面
199. * @ 参数 :
200. * @ 返回值 : 无
201. *****/
202. static void BSP_Init(void)
203. {
204.     /*
205.     * STM32中断优先级分组为4, 即4bit都用来表示抢占优先级, 范围为: 0~15
206.     * 优先级分组只需要分组一次即可, 以后如果有其他的任务需要用到中断,
207.     * 都统一用这个优先级分组, 千万不要再分组, 切忌。
208.     */
209.     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
210.
211.     /* LED 初始化 */
212.     LED_GPIO_Config();
213.
214.     /* 串口初始化 */
215.     USART_Config();
216.
217.     /* 按键初始化 */

```

```
218.     Key_GPIO_Config();
219.
220. }
221.
222. /*****END OF FILE*****/
```



六、实验现象

串口配置

端口

COM13 US1

波特率

115200

数据位

8

停止位

1

校验位

无

操作

关闭串口

这是一个FreeRTOS软件定时器实验!

这是一个FreeRTOS软件定时器实验!

Swtmr1_Callback函数执行 1 次

滴答定时器数值=1000

Swtmr1_Callback函数执行 2 次

滴答定时器数值=2000

Swtmr1_Callback函数执行 3 次

滴答定时器数值=3000

Swtmr1_Callback函数执行 4 次

滴答定时器数值=4000

Swtmr2_Callback函数执行 1 次

滴答定时器数值=5000

Swtmr1_Callback函数执行 5 次

滴答定时器数值=5004

Swtmr1_Callback函数执行 6 次

滴答定时器数值=6000

Swtmr1_Callback函数执行 7 次

滴答定时器数值=7000

Swtmr1_Callback函数执行 8 次

滴答定时器数值=8000

GSDM @不亮也很强